

internproject

January 12, 2026

```
[2]: import time
import yfinance as yf
import pandas as pd
import numpy as np
import pyarrow
import fastparquet
```

i am given a US stock lists and i try to build a factor model, at the same time, incorporate some statistical method(LDA and PCA) to improve my result.

first i need to download the stock data from 2018-01-01 to now. below is the procedure. i try to make the data easier to analyze

```
[16]: #some helper functions

def download_panel_yf(ticker_list,start="2018-01-01",end=None):
    if isinstance(ticker_list,str):
        ticker_list = [ticker_list]
    try:
        raw = yf.download(
            tickers = ticker_list,
            start = start,
            end = end,
            auto_adjust = True,
            progress = False
        )
    except Exception as e:
        print(f"Download error: {e}")
        return pd.DataFrame()

    if isinstance(raw.columns,pd.MultiIndex):
        lvl0 = set(raw.columns.get_level_values(0))
        if {"Open","High","Low","Close","Volume"}.issubset(lvl0):
            close = raw["Close"]
            vol = raw["Volume"]
        else:
            try:
                close = raw.xs("Close",level=1,axis=1)
                vol = raw.xs("Volume",level=1,axis=1)
```

```

        except KeyError:
            return pd.DataFrame()
    else:
        #one ticker
        sym = ticker_list[0]
        if "Close" not in row.columns or "Volume" not in row.columns:
            return pd.DataFrame()
        close = row[["Close"]].copy()
        close.columns = [sym]
        vol = row[["Volume"]].copy()
        vol.columns=[sym]

        #wide to long
        df_close = close.stack().rename("px_close").reset_index()
        df_close.columns = ["date", "asset", "px_close"]

        df_vol = vol.stack().rename("volume").reset_index()
        df_vol.columns=["date", "asset", "volume"]

        df = (
            df_close.merge(df_vol, on=["date", "asset"], how="inner").
            ↪sort_values(["asset", "date"]).reset_index(drop=True)
        )

        df["ret_1d"] = df.groupby("asset")["px_close"].pct_change()
        df["ret_fwd_1d"] = df.groupby("asset")["ret_1d"].shift(-1)

        df = df.dropna(subset=["ret_fwd_1d"]).reset_index(drop=True)
        return df

def get_tickers_from_excel(path, col="Symbol"):
    df = pd.read_excel(path)
    tickers = df[col].dropna().astype(str).str.strip().tolist()
    return [t.replace(".", "-") for t in tickers]

def get_sp500_tickers_online():
    try:
        url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
        table = pd.read_html(url)[0]
        tickers = table["Symbol"].tolist()
        return [t.replace(".", "-") for t in tickers]
    except Exception as e:
        print(f"failed to fetch tickers online: {e}")
        return []

def get_tickers(path=None, col="Symbol"):
    if path:

```

```

    try:
        return get_tickers_from_excel(path,col)
    except Exception as e:
        print(f"excel failed, fallback to wikipedia")
    return get_sp500_tickers_online()

#batch downloading
def download_panel_yf_batch(
    tickers,
    start = "2018-01-01",
    end = None,
    batch_size = 25,
    sleep_sec = 3
):
    all_panels = []
    bad_batches = 0
    print(f"Total tickers: {len(tickers)}")

    for i in range(0,len(tickers),batch_size):
        batch = tickers[i:i+batch_size]
        print(f"Downlaoding{i} - {i+len(batch)-1}...")
        try:
            panel = download_panel_yf(batch,start=start,end = end)
            if panel.empty:
                bad_batches+=1
            else:
                all_panels.append(panel)

        except Exception as e:
            bad_batches += 1
            print(f"batch{i} failed: {e}")
            time.sleep(sleep_sec)

    if not all_panels:
        return pd.DataFrame()

    df = pd.concat(all_panels,ignore_index=True)
    print(
        f"Done. bad batches = {bad_batches}"
        f"assets = {df['asset'].nunique()},rows={len(df)}"
    )
    return df

```

```
[17]: if __name__ == "__main__":
```

```

    EXCEL_PATH = "/Users/bohanhou/Downloads/S&P 500 tickers.xlsx"
    START_DATE = "2019-01-01"

```

```

tickers_500 = get_tickers(path=EXCEL_PATH, col="Symbol")
print(f"Tickers loaded: {len(tickers_500)}")

# test
panel_test = download_panel_yf_batch(
    tickers_500[:40],
    start=START_DATE,
    batch_size=10,
    sleep_sec=2,
)
print("TEST:", panel_test["asset"].nunique(), panel_test.shape)

# run all the tickers
panel = download_panel_yf_batch(
    tickers_500,
    start=START_DATE,
    batch_size=25,
    sleep_sec=3,
)

print("\nFinal panel:")
print("Shape:", panel.shape)
print("Unique assets:", panel["asset"].nunique())
print(panel.head())

# sanity check
print("\nPer-asset observation count:")
print(panel.groupby("asset")["date"].count().describe())

# save
panel.to_parquet("panel_sp500_2019.parquet", index=False)
print("\nSaved -> panel_sp500_2019.parquet")

```

Tickers loaded: 500

Total tickers: 40

Downlaoding0 - 9...

2 Failed downloads:

['ACE', 'AET']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

Downlaoding10 - 19...

4 Failed downloads:

['ARG', 'GAS']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['AGN', 'ALXN']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding20 - 29...

2 Failed downloads:

['ANR']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['ALTR']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding30 - 39...

3 Failed downloads:

['APOL']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['ABC', 'APC']: YFTzMissingError('possibly delisted; no timezone found')

Done. bad batches = 0assets = 29,rows=51185

TEST: 29 (51185, 6)

Total tickers: 500

Downlaoding0 - 24...

8 Failed downloads:

['ACE', 'AET', 'GAS', 'ANR', 'ARG']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['ALXN', 'AGN', 'ALTR']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding25 - 49...

3 Failed downloads:

['ABC', 'APC']: YFTzMissingError('possibly delisted; no timezone found')

['APOL']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

Downlaoding50 - 74...

6 Failed downloads:

['BRCM', 'BCR', 'BHI']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['BIG', 'AVP', 'BLL']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding75 - 99...

9 Failed downloads:

['CFN', 'CVC', 'CBG']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['COG', 'CHK', 'CBS', 'CELG', 'CERN', 'CTL']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding100 - 124...

4 Failed downloads:

['COH', 'CSC', 'CBE']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['CTXS']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding125 - 149...

9 Failed downloads:

['DNR', 'DF', 'DO', 'DTV', 'RRD', 'DFS', 'DISCA']: YFTzMissingError('possibly delisted; no timezone found')

['COV', 'CVH']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

Downlaoding150 - 174...

4 Failed downloads:

['DPS']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['DNB', 'ESV', 'ETFC']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding175 - 199...

5 Failed downloads:

['ESRX', 'FDO']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['FII', 'FLIR', 'FRX']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding200 - 224...

7 Failed downloads:

['GPS', 'HCP', 'HES', 'FTR', 'HRS']: YFTzMissingError('possibly delisted; no timezone found')

['HNZ', 'HCN']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

Downlaoding225 - 249...

4 Failed downloads:

['HSP', 'HCBK']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')

['JEC', 'IGT']: YFTzMissingError('possibly delisted; no timezone found')

Downlaoding250 - 274...

9 Failed downloads:

```
['JDSU', 'LUK', 'JOY', 'LXK', 'KFT']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')
['LIFE', 'LM', 'LLL', 'JNPR']: YFTzMissingError('possibly delisted; no timezone found')
```

Downlaoding275 - 299...

8 Failed downloads:

```
['LTD', 'MWV', 'MJN', 'LO', 'MHP', 'LLTC']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')
['MRO', 'LSI']: YFTzMissingError('possibly delisted; no timezone found')
```

Downlaoding300 - 324...

3 Failed downloads:

```
['MON', 'MYL']: YFTzMissingError('possibly delisted; no timezone found')
['MOLX']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')
```

Downlaoding325 - 349...

5 Failed downloads:

```
['NBL', 'PLL', 'JWN', 'PDCO']: YFTzMissingError('possibly delisted; no timezone found')
['NYX']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')
```

Downlaoding350 - 374...

5 Failed downloads:

```
['PBCT', 'JCP', 'PKI', 'PXD']: YFTzMissingError('possibly delisted; no timezone found')
['PCP']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')
```

Downlaoding375 - 399...

7 Failed downloads:

```
['RAI', 'SWY']: YFPricesMissingError('possibly delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')
['SAI', 'RDC', 'RTN', 'QEP', 'RHT']: YFTzMissingError('possibly delisted; no timezone found')
```

Downlaoding400 - 424...

7 Failed downloads:

```
['HOT', 'SNI', 'STJ', 'SPLS', 'SCG', 'SIAL']: YFPricesMissingError('possibly
```

```
delisted; no price data found (1d 2019-01-01 -> 2026-01-09)')
['SWN']: YFTzMissingError('possibly delisted; no timezone found')
```

Downlaoding425 - 449...

7 Failed downloads:

```
['TWX', 'TWC', 'TSO']: YFPricesMissingError('possibly delisted; no price data
found (1d 2019-01-01 -> 2026-01-09)')
['TSS', 'TIF', 'TMK', 'SYMC']: YFTzMissingError('possibly delisted; no timezone
found')
```

Downlaoding450 - 474...

7 Failed downloads:

```
['TYC', 'WAG', 'WPO']: YFPricesMissingError('possibly delisted; no price data
found (1d 2019-01-01 -> 2026-01-09)')
['VAR', 'X', 'VIAB', 'UTX']: YFTzMissingError('possibly delisted; no timezone
found')
```

Downlaoding475 - 499...

9 Failed downloads:

```
['ZMH', 'WFM', 'WYN', 'WLP']: YFPricesMissingError('possibly delisted; no price
data found (1d 2019-01-01 -> 2026-01-09)')
['WIN', 'WPX', 'XLNX', 'XL', 'YH00']: YFTzMissingError('possibly delisted; no
timezone found')
```

Done. bad batches = 0assets = 373,rows=627560

Final panel:

Shape: (627560, 6)

Unique assets: 373

	date	asset	px_close	volume	ret_1d	ret_fwd_1d
0	2019-01-02	A	62.442287	2113300.0	NaN	-0.036840
1	2019-01-03	A	60.141933	5383900.0	-0.036840	0.034614
2	2019-01-04	A	62.223671	3123700.0	0.034614	0.021234
3	2019-01-07	A	63.544937	3235100.0	0.021234	0.014660
4	2019-01-08	A	64.476486	1578100.0	0.014660	0.020935

Per-asset observation count:

count	373.000000
mean	1682.466488
std	326.249735
min	29.000000
25%	1765.000000
50%	1765.000000
75%	1765.000000
max	1765.000000

Name: date, dtype: float64

Saved -> panel_sp500_2019.parquet

by now all the available data saved, we do some data cleaning here

```
[3]: panel = pd.read_parquet("/Users/bohanhou/PycharmProjects/ML Project/
    ↪panel_sp500_2019.parquet")
print(panel)
obs = panel.groupby("asset")["date"].count()
print(obs)
MIN_DAYS = 252

good_assets = obs[obs>=MIN_DAYS].index
print(good_assets)
panel_u = panel[panel["asset"].isin(good_assets)].copy()
print("after min_days filter")
print(panel_u["asset"].nunique(), panel_u.shape)

panel_u = panel_u[panel_u["px_close"]>=1.0]
panel_u = panel_u[panel_u["ret_1d"].abs()<0.5]
universe = sorted(panel_u["asset"].unique())
print("final universe size: ", len(universe))
print(panel_u.groupby("asset")["date"].count().describe())
panel_u.to_parquet("panel_sp500_clean.parquet")
```

	date	asset	px_close	volume	ret_1d	ret_fwd_1d
0	2019-01-02	A	62.442287	2113300.0	NaN	-0.036840
1	2019-01-03	A	60.141933	5383900.0	-0.036840	0.034614
2	2019-01-04	A	62.223671	3123700.0	0.034614	0.021234
3	2019-01-07	A	63.544937	3235100.0	0.021234	0.014660
4	2019-01-08	A	64.476486	1578100.0	0.014660	0.020935
...	...	...	...	...	...	...
627555	2026-01-02	ZION	59.250000	1194000.0	0.012128	0.017890
627556	2026-01-05	ZION	60.310001	1615000.0	0.017890	0.010778
627557	2026-01-06	ZION	60.959999	1436100.0	0.010778	-0.013287
627558	2026-01-07	ZION	60.150002	1026600.0	-0.013287	0.011139
627559	2026-01-08	ZION	60.820000	1285700.0	0.011139	-0.009701

[627560 rows x 6 columns]

asset

A	1765
AA	1765
AAPL	1765
ABT	1765
ACN	1765
...	...
XRAY	1765

```

XRX      1765
XYL      1765
YUM      1765
ZION     1765
Name: date, Length: 373, dtype: int64
Index(['A', 'AA', 'AAPL', 'ABT', 'ACN', 'ADBE', 'ADI', 'ADM', 'ADP', 'ADSK',
      ...
      'WU', 'WY', 'WYNN', 'XEL', 'XOM', 'XRAY', 'XRX', 'XYL', 'YUM', 'ZION'],
      dtype='object', name='asset', length=362)
after min_days filter
362 (626436, 6)
final universe size: 361
count      361.000000
mean       1732.407202
std        164.588655
min        516.000000
25%        1764.000000
50%        1764.000000
75%        1764.000000
max        1764.000000
Name: date, dtype: float64

```

Build some fundamental factors and calculate their IC

```

[9]: panel = pd.read_parquet("/Users/bohanhou/PycharmProjects/ML Project/
    ↪panel_sp500_clean.parquet")

panel = panel.sort_values(["asset", "date"])
print(panel.head())
g = panel.groupby("asset", group_keys=False)
print(g)
#momentum factors
panel["f_mom_20"] = g["ret_1d"].apply(
    lambda x: x.rolling(20, min_periods=20).sum()
)
panel["f_mom_5"] = g["ret_1d"].apply(
    lambda x: x.rolling(5, min_periods=5).sum()
)
#volatility
panel["f_vol_20"] = g["ret_1d"].apply(
    lambda x: x.rolling(20, min_periods=20).std()
)
#volume spike factors
panel["vol_ma20"] = g["volume"].apply(
    lambda x: x.shift(1).rolling(20, min_periods = 20).mean()
)
panel["f_vol_5"] = g["volume"].apply(
    lambda x: x.rolling(5, min_periods=5).std()
)

```

```

)
panel["f_vol_spike"]=panel["volume"]/panel["vol_ma20"]

#rev factor
panel["f_rev_1"]=-panel["ret_1d"]

#ma gap factor
panel["px_ma20"]=g["px_close"].apply(
    lambda x:x.rolling(20,min_periods=20).mean()
)
panel["f_ma_gap"]=panel["px_close"]/panel["px_ma20"]-1

factor_cols = [
    "f_mom_5",
    "f_mom_20",
    "f_vol_20",
    "f_vol_spike",
    "f_rev_1",
    "f_ma_gap",
]

panel_f = panel[["date","asset","ret_fwd_1d"]+factor_cols].copy()

#winsorization to remove outliers
def winsorize_cs(df,cols,p=0.01):
    def _w(s):
        lo,hi = s.quantile(p),s.quantile(1-p)
        return s.clip(lo,hi)
    for c in cols:
        df[c] = df.groupby("date")[c].transform(_w)
    return df
panel_f = winsorize_cs(panel_f,factor_cols,p=0.01)

#rank the factor within each day
for c in factor_cols:
    panel_f[c] = panel_f.groupby("date")[c].rank(pct=True)

#calculate IC
def daily_rank_ic(df, factor, ret="ret_fwd_1d"):
    tmp = df[["date", factor, ret]].dropna()
    return tmp.groupby("date")[["factor", ret]].apply(
        lambda x: x[factor].corr(x[ret], method="spearman")
    )

ic_summary={}
for c in factor_cols:

```

```

ic = daily_rank_ic(panel_f,c)
ic_summary[c]={
    "IC_mean": ic.mean(),
    "IC_std": ic.std(),
    "ICIR": ic.mean()/ic.std() if ic.std()>0 else np.nan,
    "hit_rate":(ic>0).mean()
}
print(pd.DataFrame(ic_summary))

```

	date	asset	px_close	volume	ret_1d	ret_fwd_1d
1	2019-01-03	A	60.141933	5383900.0	-0.036840	0.034614
2	2019-01-04	A	62.223671	3123700.0	0.034614	0.021234
3	2019-01-07	A	63.544937	3235100.0	0.021234	0.014660
4	2019-01-08	A	64.476486	1578100.0	0.014660	0.020935
5	2019-01-09	A	65.826294	2442300.0	0.020935	0.009386

<pandas.core.groupby.generic.DataFrameGroupBy object at 0x125295f10>

	f_mom_5	f_mom_20	f_vol_20	f_vol_spike	f_rev_1	f_ma_gap
IC_mean	-0.012901	-0.006931	-0.005031	0.009183	0.015544	-0.011614
IC_std	0.209101	0.212584	0.270465	0.083688	0.203624	0.215498
ICIR	-0.061696	-0.032601	-0.018603	0.109733	0.076338	-0.053895
hit_rate	0.485795	0.497421	0.485960	0.551606	0.540249	0.484814

Incorporate some statistical methods

```

[38]: df = panel_f.copy()
df["y"] = df.groupby("date")["ret_fwd_1d"].transform(lambda s: s>s.median()).
    ↪astype(int)

dates = np.sort(df["date"].unique())
print(df)
split = dates[int(len(dates)*0.7)]

train = df[df["date"]<=split].copy()
test = df[df["date"]>split].copy()

#calculate weights of LDA
mu_up = train.loc[train["y"]==1,factor_cols].mean()
mu_dn = train.loc[train["y"]==0,factor_cols].mean()

w = (mu_up-mu_dn).values
print(w)
w = w/np.linalg.norm(w)

w_series= pd.Series(w,index=factor_cols).sort_values(ascending=False)
print("LDA weights\n", w_series)

```

```

#so we can create the LDA factor column in the original factor, using weight
↳times the original factors, we call it ic_lda

for d in (train,test):
    d["alpha_lda"] = d[factor_cols].values@w
    d["alpha_lda"] = d.groupby("date")["alpha_lda"].rank(pct=True)

#evaluate on test
ic_lda = daily_rank_ic(test,"alpha_lda")
result = {
    "IC_mean": float(ic_lda.mean()),
    "IC_std": float(ic_lda.std(ddof=1)),
    "ICIR": float(ic_lda.mean() / ic_lda.std(ddof=1)) if ic_lda.std(ddof=1) > 0
↳else np.nan,
    "hit_rate": float((ic_lda > 0).mean()),
    "n_days": int(ic_lda.dropna().shape[0]),
}
print(f"LDA result {result}")

```

	date	asset	ret_fwd_1d	f_mom_5	f_mom_20	f_vol_20	\
1	2019-01-03	A	0.034614	NaN	NaN	NaN	
2	2019-01-04	A	0.021234	NaN	NaN	NaN	
3	2019-01-07	A	0.014660	NaN	NaN	NaN	
4	2019-01-08	A	0.020935	NaN	NaN	NaN	
5	2019-01-09	A	0.009386	0.699140	NaN	NaN	
...	...	...	...	...	...	...	
627555	2026-01-02	ZION	0.017890	0.460674	0.904494	0.536517	
627556	2026-01-05	ZION	0.010778	0.654494	0.842697	0.508427	
627557	2026-01-06	ZION	-0.013287	0.660112	0.853933	0.435393	
627558	2026-01-07	ZION	0.011139	0.668539	0.817416	0.457865	
627559	2026-01-08	ZION	-0.009701	0.597183	0.766197	0.433803	

	f_vol_spike	f_rev_1	f_ma_gap	y
1	NaN	0.842407	NaN	1
2	NaN	0.469914	NaN	1
3	NaN	0.174785	NaN	1
4	NaN	0.343840	NaN	1
5	NaN	0.146132	NaN	1
...	...	...	...	...
627555	0.480337	0.407303	0.634831	1
627556	0.497191	0.323034	0.693820	1
627557	0.376404	0.452247	0.626404	0
627558	0.089888	0.561798	0.603933	0
627559	0.290141	0.526761	0.538028	0

[625399 rows x 10 columns]

[-0.00632623 -0.00206087 -0.00383965 0.00446798 0.00839895 -0.00389434]

```
LDA weights
  f_rev_1      0.654487
  f_vol_spike   0.348167
  f_mom_20     -0.160593
  f_vol_20     -0.299204
  f_ma_gap     -0.303466
  f_mom_5      -0.492971
dtype: float64
LDA result {'IC_mean': 0.018762339304508678, 'IC_std': 0.16667759298580298,
'ICIR': 0.11256665619179411, 'hit_rate': 0.553875236294896, 'n_days': 529}
```

Now we have the LDA factor in the training set the lda result gives me some information, $ic > 0$, which is not bad but the ic std is bad ,icir around 0.11, which means there is a lot of noise and my single LDA signal is is not stable. however, i still wanna do a back test for it

```
[31]: group1 = test[["date","asset","alpha_lda","ret_fwd_1d"]]
      group2= train[["date","asset","alpha_lda","ret_fwd_1d"]]

      group = pd.concat([group1,group2])
      group.to_parquet("alpha_data")

      s = group["alpha_lda"]

      #if we do long short trading and equal weighted , we can do something like this
      lo = s.quantile(0.2)
      hi = s.quantile(0.8)
      long = group[group["alpha_lda"]>=hi]
      short = group[group["alpha_lda"]<=lo]
      # quick sanity display
      print(test[test["asset"]=="AAPL"][["date","asset","alpha_lda","ret_fwd_1d"]].
        ↪tail(10))
```

	date	asset	alpha_lda	ret_fwd_1d
31760	2025-12-24	AAPL	0.808989	-0.001497
31761	2025-12-26	AAPL	0.907303	0.001317
31762	2025-12-29	AAPL	0.665730	-0.002484
31763	2025-12-30	AAPL	0.710674	-0.004468
31764	2025-12-31	AAPL	0.601124	-0.003127
31765	2026-01-02	AAPL	0.884831	-0.013837
31766	2026-01-05	AAPL	0.955056	-0.018334
31767	2026-01-06	AAPL	1.000000	-0.007737
31768	2026-01-07	AAPL	0.921348	-0.004955
31769	2026-01-08	AAPL	0.997183	0.001274

Now we can do some long short trade based on our data, and we trade based on signal weighting(stronger signal with higher position and vice versa). we keep dollar neutral in this case.

```
[37]: data = pd.read_parquet("/Users/bohanhou/PycharmProjects/ML Project/alpha_data")
      data["date"] = pd.to_datetime(data["date"])
```

```

data = data.sort_values(["date", "asset"]).reset_index(drop = True)

#performance metrics

def perf_stat(daily_ret, ann=252):
    daily_ret = daily_ret.dropna()
    mu = daily_ret.mean()
    sig = daily_ret.std(ddof=1)
    sharpe = (mu/sig)*np.sqrt(ann) if sig>0 else np.nan

    equity = (1+daily_ret).cumprod()
    peak = equity.cummax()
    dd = equity/peak -1 #max drawdown
    mdd = dd.min()

    cagr = equity.iloc[-1]**(ann/len(daily_ret)) - 1 if len(daily_ret)>0 else
    np.nan
    vol_ann = sig*np.sqrt(ann)
    return {
        "n_days": int(len(daily_ret)),
        "mean_daily": float(mu),
        "vol_daily": float(sig),
        "sharpe_ann": float(sharpe),
        "cagr_est": float(cagr),
        "vol_ann": float(vol_ann),
        "max_drawdown": float(mdd),
    }

print(df)

# we add a column to represent the alpha we just build, at the same time we
    smooth it to make sure to avoid noisy signal

def add_alpha_smooth(df, alpha_col="alpha_lda", smooth_window=5):
    df = df.sort_values(["asset", "date"]).copy()
    df["alpha_smooth"] = (
        df.groupby("asset")[alpha_col]
        .rolling(smooth_window, min_periods=smooth_window)
        .mean()
        .reset_index(level=0, drop=True) # drop asset
    )
    return df

#build daily weights, for each day, calculate the crosssectional z score, clip
    extreme z, demean, normalize to make sure dollar-neutral
def build_ls_weights_quantile(
    df,

```

```

signal_col="alpha_smooth",
q=0.2,
clip_z=3.0,
):
    x = df[["date", "asset", signal_col]].copy()

    def one_day(g):
        s = g[signal_col].astype(float)
        out = g[["date", "asset"]].copy()
        out["w"] = 0.0
        s = s.dropna()
        if s.empty:
            return out

        lo = s.quantile(q)
        hi = s.quantile(1 - q)

        long_idx = s[s >= hi].index
        short_idx = s[s <= lo].index
        trade_idx = long_idx.union(short_idx)

        if len(long_idx) == 0 or len(short_idx) == 0:
            return out
        st = s.loc[trade_idx]
        mu = st.mean()
        mu = st.mean()
        sd = st.std(ddof=1)
        if sd == 0 or np.isnan(sd):
            return out

        z = (st - mu) / sd
        z = z.clip(-clip_z, clip_z)
        z = z - z.mean()
        denom = np.abs(z).sum()
        if denom > 0:
            z = z / denom
            out.loc[z.index, "w"] = z.values

        return out
    return x.groupby("date", group_keys=False).apply(one_day).
↪reset_index(drop=True)

#now we can calculate our return or pnl
def portfolio_pnl(df,wdf,ret_col="ret_fwd_1d",cost_bps=0.0):

```

```

x = df[["date", "asset", ret_col]].dropna().copy()
x = x.merge(wdf, on=["date", "asset"], how="inner").
↳ dropna(subset=["w", ret_col])
daily_gross = x.groupby("date").apply(lambda g: np.sum(g["w"].values *
↳ g[ret_col].values))
daily_gross = daily_gross.rename("gross").to_frame()
w_only = wdf.copy().sort_values(["asset", "date"])
w_only["w_prev"] = w_only.groupby("asset")["w"].shift(1)

# For turnover at date t, we need abs(w_t - w_{t-1}) across assets that
↳ exist both days.
w_only["abs_dw"] = (w_only["w"] - w_only["w_prev"]).abs()
daily_turn = w_only.groupby("date")["abs_dw"].sum() * 0.5
# 0.5 makes it "one-way" turnover
daily_turn = daily_turn.rename("turnover").to_frame()

out = daily_gross.join(daily_turn, how="left")
out["turnover"] = out["turnover"].fillna(0.0)

cost = cost_bps / 10000.0
out["cost"] = cost * out["turnover"]
out["net"] = out["gross"] - out["cost"]

return out.reset_index()

SMOOTH_W = 5      # try 3,5,10 later
CLIP_Z = 3.0      # z-score clipping to reduce outliers
COST_BPS = 10.0   # realistic-ish starting point (you can test 5/10/20)

data2 = add_alpha_smooth(data, alpha_col="alpha_lda", smooth_window=SMOOTH_W)

wdf = build_ls_weights_quantile(data2, signal_col="alpha_smooth", clip_z=CLIP_Z)

daily = portfolio_pnl(data2, wdf, ret_col="ret_fwd_1d", cost_bps=COST_BPS)

print("Weights sanity:")
print(wdf.groupby("date")["w"].apply(lambda s: pd.Series({
    "sum_w": s.sum(),
    "sum_abs_w": np.abs(s).sum(),
    "n": s.size
})).head())

print("\nTurnover summary:")
print(daily["turnover"].describe())

print("\nGROSS stats:")
print(perf_stat(daily["gross"]))

```

```
print(f"\nNET stats (cost={COST_BPS} bps * turnover):")
print(perf_stat(daily["net"]))

daily.to_parquet("weighted_ls_daily.parquet", index=False)
print("\nSaved -> weighted_ls_daily.parquet")
```

	date	asset	ret_fwd_1d	f_mom_5	f_mom_20	f_vol_20	\
1	2019-01-03	A	0.034614	NaN	NaN	NaN	
2	2019-01-04	A	0.021234	NaN	NaN	NaN	
3	2019-01-07	A	0.014660	NaN	NaN	NaN	
4	2019-01-08	A	0.020935	NaN	NaN	NaN	
5	2019-01-09	A	0.009386	0.699140	NaN	NaN	
...	...	...	...	...	...	...	
627555	2026-01-02	ZION	0.017890	0.460674	0.904494	0.536517	
627556	2026-01-05	ZION	0.010778	0.654494	0.842697	0.508427	
627557	2026-01-06	ZION	-0.013287	0.660112	0.853933	0.435393	
627558	2026-01-07	ZION	0.011139	0.668539	0.817416	0.457865	
627559	2026-01-08	ZION	-0.009701	0.597183	0.766197	0.433803	

	f_vol_spike	f_rev_1	f_ma_gap	y
1	NaN	0.842407	NaN	1
2	NaN	0.469914	NaN	1
3	NaN	0.174785	NaN	1
4	NaN	0.343840	NaN	1
5	NaN	0.146132	NaN	1
...	...	...	...	...
627555	0.480337	0.407303	0.634831	1
627556	0.497191	0.323034	0.693820	1
627557	0.376404	0.452247	0.626404	0
627558	0.089888	0.561798	0.603933	0
627559	0.290141	0.526761	0.538028	0

[625399 rows x 10 columns]

```
/var/folders/tt/2zmln3691579pyj9pfcgqgtm0000gn/T/ipykernel_60697/3491576815.py:8
6: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This
behavior is deprecated, and in a future version of pandas the grouping columns
will be excluded from the operation. Either pass `include_groups=False` to
exclude the groupings or explicitly select the grouping columns after groupby to
silence this warning.
```

```
return x.groupby("date",
group_keys=False).apply(one_day).reset_index(drop=True)
/var/folders/tt/2zmln3691579pyj9pfcgqgtm0000gn/T/ipykernel_60697/3491576815.py:9
5: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This
behavior is deprecated, and in a future version of pandas the grouping columns
will be excluded from the operation. Either pass `include_groups=False` to
exclude the groupings or explicitly select the grouping columns after groupby to
```

```
silence this warning.
```

```
daily_gross = x.groupby("date").apply(lambda g: np.sum(g["w"].values *  
g[ret_col].values))
```

```
Weights sanity:
```

```
date
```

```
2019-01-03  sum_w          0.0  
            sum_abs_w     0.0  
            n             349.0  
2019-01-04  sum_w          0.0  
            sum_abs_w     0.0
```

```
Name: w, dtype: float64
```

```
Turnover summary:
```

```
count    1764.000000  
mean      0.204046  
std       0.043240  
min       0.000000  
25%       0.181282  
50%       0.205344  
75%       0.229566  
max       0.500000
```

```
Name: turnover, dtype: float64
```

```
GROSS stats:
```

```
{'n_days': 1764, 'mean_daily': -8.827002357787243e-07, 'vol_daily':  
0.0074267327747139, 'sharpe_ann': -0.0018867558940911671, 'cagr_est':  
-0.007257177818482896, 'vol_ann': 0.117895727853754, 'max_drawdown':  
-0.28780957453097067}
```

```
NET stats (cost=10.0 bps * turnover):
```

```
{'n_days': 1764, 'mean_daily': -0.00020492914638733093, 'vol_daily':  
0.007426202159023529, 'sharpe_ann': -0.43806366655732487, 'cagr_est':  
-0.057020342442058, 'vol_ann': 0.11788730459080318, 'max_drawdown':  
-0.39022348760445824}
```

```
Saved -> weighted_ls_daily.parquet
```

as i expected, The LDA signal exhibits statistically significant cross-sectional information, with a positive mean IC and a hit rate above 50%. However, its daily ICIR is very low, indicating a poor signal-to-noise ratio at the daily horizon. When traded with daily rebalancing in a 20/20 long-short portfolio, the strategy generates essentially zero gross return, while transaction costs driven by non-trivial turnover lead to significantly negative net performance. This suggests that the alpha is not absent, but is too weak and unstable to be monetized at a daily frequency. Temporal aggregation, longer holding periods, or reduced rebalancing frequency are required to align the trading structure with the persistence of the underlying signal.

```

[59]: import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.covariance import LedoitWolf

# IC function (Spearman rank IC per day)

def daily_rank_ic(df, factor, ret="ret_fwd_1d"):
    return (
        df[["date", factor, ret]]
        .dropna()
        .groupby("date")
        .apply(lambda x: x[factor].corr(x[ret], method="spearman"))
    )

def ic_stats(ic):
    ic = ic.dropna()
    return {
        "IC_mean": float(ic.mean()),
        "IC_std": float(ic.std(ddof=1)),
        "ICIR": float(ic.mean()/ic.std(ddof=1)) if ic.std(ddof=1) > 0 else np.
↪nan,
        "hit_rate": float((ic > 0).mean()),
        "n_days": int(ic.shape[0]),
    }

# Cross-sectional zscore (ONLY for PCA step)

def cs_zscore(df, cols):
    out = df.copy()
    for c in cols:
        mu = out.groupby("date")[c].transform("mean")
        sd = out.groupby("date")[c].transform("std")
        out[c] = (out[c] - mu) / sd
    return out

# Daily PC1 scores on z-scored factors
def daily_pcl(df, factor_cols, min_assets=None):
    """
    df must contain: ["date", "asset"] + factor_cols
    returns: Series PC1 aligned to df index (NaN where insufficient data)
    """
    k = len(factor_cols)
    if min_assets is None:
        min_assets = max(k + 1, 30)

```

```

pc1 = pd.Series(np.nan, index=df.index, name="PC1")

for d, g in df.groupby("date"):
    g_valid = g.dropna(subset=factor_cols)
    if g_valid.shape[0] < min_assets:
        continue
    X = g_valid[factor_cols].values
    pca = PCA(n_components=1)
    z = pca.fit_transform(X).ravel()
    pc1.loc[g_valid.index] = z

return pc1

# Neutralize each factor to PC1 per date: f_resid = f - beta*PC1

def neutralize_factors_to_pc1(df, factor_cols, pc1_col="PC1"):
    out = df.copy()
    for c in factor_cols:
        def one_day(g):
            x = g[pc1_col].astype(float)
            y = g[c].astype(float)
            mask = x.notna() & y.notna()
            if mask.sum() < 5:
                return y
            xv, yv = x[mask].values, y[mask].values
            vx = np.var(xv)
            if vx <= 0 or np.isnan(vx):
                return y
            beta = np.cov(xv, yv, ddof=1)[0, 1] / vx
            y_res = y.copy()
            y_res.loc[mask] = y.loc[mask] - beta * x.loc[mask]
            return y_res

        out[c + "_resid"] = out.groupby("date", group_keys=False).apply(one_day)
    return out

# previous "LDA" weights (mean difference) + alpha + daily rank

def mean_diff_alpha(train, test, feature_cols, label_col="y"):
    # compute weights on train
    mu_up = train.loc[train[label_col] == 1, feature_cols].mean()
    mu_dn = train.loc[train[label_col] == 0, feature_cols].mean()

    w = (mu_up - mu_dn).values.astype(float)
    norm = np.linalg.norm(w)

```

```

if norm == 0 or np.isnan(norm):
    w = np.zeros_like(w)
else:
    w = w / norm

# score both train/test, then daily rank (pct=True)
for d in (train, test):
    d["alpha"] = d[feature_cols].values @ w
    d["alpha"] = d.groupby("date")["alpha"].rank(pct=True)

return w, train, test

#
# RUN: exact replication + PCA-residual variant

def true_lda_alpha(train, test, feature_cols, label_col="y"):
    """
    Same pipeline as your meanDiff baseline,
    but  $w = \Sigma^{-1}(\mu_{up} - \mu_{dn})$  with LedoitWolf shrinkage.
    Robust to NaN/inf by filtering valid rows in TRAIN only.
    """

    # make sure numeric + finite on TRAIN
    tr = train.copy()

    # force numeric (if any columns are object)
    for c in feature_cols:
        tr[c] = pd.to_numeric(tr[c], errors="coerce")

    # keep only finite rows for covariance/means
    Xtr = tr[feature_cols].replace([np.inf, -np.inf], np.nan)
    mask = Xtr.notna().all(axis=1) & tr[label_col].notna()
    tr = tr.loc[mask].copy()
    X = tr[feature_cols].values

    # safety: need enough rows + both classes
    if tr.shape[0] < 50 or tr[label_col].nunique() < 2:
        w = np.zeros(len(feature_cols), dtype=float)
        for d in (train, test):
            d["alpha"] = np.nan
        return w, train, test

    # class means (on cleaned TRAIN)
    mu_up = tr.loc[tr[label_col] == 1, feature_cols].mean().values
    mu_dn = tr.loc[tr[label_col] == 0, feature_cols].mean().values
    dm_u = mu_up - mu_dn

```

```

# shrinkage covariance on cleaned TRAIN
lw = LedoitWolf().fit(X)
Sigma_inv = np.linalg.pinv(lw.covariance_) # pinv is safer than inv

# true LDA direction
w = Sigma_inv @ dmu
norm = np.linalg.norm(w)
if norm > 0:
    w = w / norm

# score + daily rank (apply to original train/test; NaNs naturally
↳propagate)
for d in (train, test):
    # ensure numeric in scoring data too
    for c in feature_cols:
        d[c] = pd.to_numeric(d[c], errors="coerce")
    Xd = d[feature_cols].replace([np.inf, -np.inf], np.nan).values
    d["alpha"] = Xd @ w
    d["alpha"] = d.groupby("date")["alpha"].rank(pct=True)

return w, train, test

def run_raw_vs_pca(panel_f, factor_cols):
    df = panel_f.copy()
    df["date"] = pd.to_datetime(df["date"])

    # label exactly as you did
    df["y"] = df.groupby("date")["ret_fwd_1d"].transform(lambda s: s > s.
↳median()).astype(int)

    dates = np.sort(df["date"].unique())
    split = dates[int(len(dates) * 0.7)]

    train0 = df[df["date"] <= split].copy()
    test0 = df[df["date"] > split].copy()

    # (A) RAW mean-diff baseline (your original)

    w_raw, train_raw, test_raw = mean_diff_alpha(
        train0.copy(), test0.copy(), factor_cols, label_col="y"
    )
    ic_raw = daily_rank_ic(test_raw, "alpha", ret="ret_fwd_1d")
    stats_raw = ic_stats(ic_raw)

    # (A2) TRUE LDA with shrinkage:  $w = \Sigma^{-1}(\mu_{up} - \mu_{dn})$ 

```

```

w_true, train_true, test_true = true_lda_alpha(
    train0.copy(), test0.copy(), factor_cols, label_col="y"
)
ic_true = daily_rank_ic(test_true, "alpha", ret="ret_fwd_1d")
stats_true = ic_stats(ic_true)

# (B) PCA-residual + mean-diff (prior PCA residual test)

df_pca = df.copy()

# z-scored factors only to compute PC1
df_for_pca = cs_zscore(df_pca[["date", "asset"] + factor_cols].copy(),
↪factor_cols)
df_pca["PC1"] = daily_pc1(df_for_pca[["date", "asset"] + factor_cols],
↪factor_cols)

# neutralize ORIGINAL factors to PC1
df_pca = neutralize_factors_to_pc1(
    df_pca[["date", "asset", "ret_fwd_1d", "y", "PC1"] + factor_cols].
↪copy(),
    factor_cols,
    pc1_col="PC1"
)
resid_cols = [c + "_resid" for c in factor_cols]

train1 = df_pca[df_pca["date"] <= split].copy()
test1 = df_pca[df_pca["date"] > split].copy()

w_pca, train_pca, test_pca = mean_diff_alpha(
    train1, test1, resid_cols, label_col="y"
)
ic_pca = daily_rank_ic(test_pca, "alpha", ret="ret_fwd_1d")
stats_pca = ic_stats(ic_pca)

# Output table + weights

table = pd.DataFrame({
    "RAW_meanDiff": stats_raw,
    "TRUE_LDA_shrink": stats_true,
    "PCAresid_meanDiff": stats_pca,
}).T

w_raw_series = pd.Series(w_raw, index=factor_cols).
↪sort_values(ascending=False)

```

```

    w_true_series = pd.Series(w_true, index=factor_cols).
↪sort_values(ascending=False)
    w_pca_series = pd.Series(w_pca, index=resid_cols).
↪sort_values(ascending=False)

    return table, w_raw_series, w_true_series, w_pca_series, split

# Usage:
table, w_raw, w_true, w_pca, split = run_raw_vs_pca(panel_f, factor_cols)
print("split date:", split)
print(table)
# print("\nRAW weights:\n", w_raw)
# print("\nPCA-resid weights:\n", w_pca)

```

```

/var/folders/tt/2zmln3691579pyj9pfcgqgtm0000gn/T/ipykernel_60697/3611414659.py:1
0: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This
behavior is deprecated, and in a future version of pandas the grouping columns
will be excluded from the operation. Either pass `include_groups=False` to
exclude the groupings or explicitly select the grouping columns after groupby to
silence this warning.

```

```
df[["date", factor, ret]]
```

```

/var/folders/tt/2zmln3691579pyj9pfcgqgtm0000gn/T/ipykernel_60697/3611414659.py:1
0: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This
behavior is deprecated, and in a future version of pandas the grouping columns
will be excluded from the operation. Either pass `include_groups=False` to
exclude the groupings or explicitly select the grouping columns after groupby to
silence this warning.

```

```
df[["date", factor, ret]]
```

```

/var/folders/tt/2zmln3691579pyj9pfcgqgtm0000gn/T/ipykernel_60697/3611414659.py:8
4: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This
behavior is deprecated, and in a future version of pandas the grouping columns
will be excluded from the operation. Either pass `include_groups=False` to
exclude the groupings or explicitly select the grouping columns after groupby to
silence this warning.

```

```
out[c + "_resid"] = out.groupby("date", group_keys=False).apply(one_day)
```

```

/var/folders/tt/2zmln3691579pyj9pfcgqgtm0000gn/T/ipykernel_60697/3611414659.py:8
4: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This
behavior is deprecated, and in a future version of pandas the grouping columns
will be excluded from the operation. Either pass `include_groups=False` to
exclude the groupings or explicitly select the grouping columns after groupby to
silence this warning.

```

```
out[c + "_resid"] = out.groupby("date", group_keys=False).apply(one_day)
```

```

/var/folders/tt/2zmln3691579pyj9pfcgqgtm0000gn/T/ipykernel_60697/3611414659.py:8
4: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This
behavior is deprecated, and in a future version of pandas the grouping columns
will be excluded from the operation. Either pass `include_groups=False` to
exclude the groupings or explicitly select the grouping columns after groupby to

```


longer holding periods and reduced rebalancing frequency to mitigate transaction costs, as well as alpha-level neutralization to common components instead of factor-level PCA, which was shown to remove predictive information. Additional directions include experimenting with more selective labeling schemes for LDA (e.g., top/bottom quantiles) to improve signal-to-noise ratio, and allowing LDA weights to evolve over time via rolling estimation to account for regime dependence. These extensions aim to improve monetization of the existing alpha without altering its underlying economic structure.